

Callout 2.1 Formal problem definition: the true utility function $\mathcal{U}$ must be simplified to make it optimizable in practice.

Consider an EW system with N heterogeneous nodes. Each node $n \in N$ has the following:

- A set of $\bar{o}_n$ observable features, denoted $\mathbf{o}_n \triangleq (o_{n1}, o_{n2}, \dots, o_{n\bar{o}_n})$.
- A set of $\bar{c}_n$ control parameters, denoted $\mathbf{c}_n \triangleq (c_{n1}, c_{n2}, \dots, c_{n\bar{c}_n})$.

Denote unobservable contextual information as $\mathbf{z}$.

To capture changes over time, denote $\mathbf{c}_n(t)$ to be the value of c_n at time t , and do likewise for $\mathbf{o}_n$ and $\mathbf{z}$.

Associated with the system is a real-valued scalar utility measure $\mathcal{U}(t)$ that characterizes the global, network-wide performance measure at time t . This measure is a function $\mathcal{F}$ of all the control parameters, observable features, and unobservable factors, for all nodes, since the beginning of the mission:

$$\mathcal{U}(t+1) = \mathcal{F}(\forall n \in N(\mathbf{o}_n(0), \dots, \mathbf{o}_n(t), \mathbf{c}_n(0), \dots, \mathbf{c}_n(t), \mathbf{z}(0), \dots, \mathbf{z}(t)))$$

The goal is to solve the following distributed optimization problem:

Design a fully distributed algorithm where every node n determines its control parameter values $\mathbf{c}_n(t)$ using only its own previous observable features $\mathbf{o}_n(0), \dots, \mathbf{o}_n(t)$ and control values $\mathbf{c}_n(0), \dots, \mathbf{c}_n(t)$ such that $\mathcal{U}(t+1)$ is maximized (or minimized) for each t .*

The challenge is that we cannot define an exact analytical expression for $\mathcal{F}$, due to the unobservable factors $\mathbf{z}$ and complex intra- and internode interactions. Therefore, the algorithms described in this book and in the related work generally simplify the problem by using local, memory-less approximations of $\mathcal{F}$: Each node approximates the true utility $\mathcal{U}$ with $\tilde{\mathcal{U}}_n$.

$$\begin{aligned} \tilde{\mathcal{U}}_n(t+1) &= \tilde{\mathcal{F}}_n(\mathbf{o}_n(t), \mathbf{c}_n(t)) \\ \forall n \in N, \mathcal{U}(t) &\approx \tilde{\mathcal{U}}_n(t) \end{aligned}$$

Essentially, node n assumes that decisions made historically ($\mathbf{c}_n(t' < t)$), and by nearby nodes ($n' \neq n$), will be *implicitly* observable in $\mathbf{o}_n(t)$. For

* The optimization goal can be expanded to compute the Utility of the entire mission, rather than for each consecutive instantaneous time value t .

example, if a neighbor n' increases data rate, node n will observe increased congestion. When nodes $n' \neq n$ *explicitly* share the previous observables or control settings with node n [i.e., $\mathbf{o}_{n'} \neq \mathbf{o}_n(t' < t)$ or $\mathbf{c}_{n'} \neq \mathbf{c}_n(t' < t)$], these values become additional features in $\mathbf{o}_n(t)$.

To configure the nodes, at each timestep t , each node selects a strategy $\mathbf{s}_n(t)$ that is a particular setting of controllables $\mathbf{c}_n(t)$ that optimize performance on this surface:

$$\mathbf{s}_n(t) = \arg \max_{\mathbf{c}_n(t)} \tilde{\mathcal{F}}_n(\mathbf{o}_n(t), \mathbf{c}_n(t))$$

(Or alternatively, $\arg \min$.) There are $\prod_{v \in v_{c_n}} v_{c_n}$ candidate strategies for node n at each timestep, where v_{c_n} is the number of values that a given controllable c_n can assume. This number becomes infinity when controllables are continuous-valued.

To support changing the system's behavior during the mission, we express $\tilde{U}_n$ as a set of $\overline{m}_n$ metrics, m_{nk} , allowing a user to change corresponding weights w_n during the mission.[†] Thus for a node $n \in N$, $\tilde{U}_n(t)$ is a function of its metrics and their weights, and each metric is in turn a function of the observables and controllables at that node:

$$\begin{aligned} m_{nk}(t+1) &= f_{nk}(\mathbf{o}_n(t), \mathbf{c}_n(t)) \\ v_{nk}(t) &= (w_{nk}(t), m_{nk}(t)) \\ \tilde{U}_n(t) &= g_n(v_{n1}(t), v_{n2}(t), \dots, v_{n\overline{m}_n}(t)) \end{aligned}$$

The functions f_{nk} are models created by hand or empirically learned. The model f_{nk} at time t estimates the metric value $m_{nk}(t+1)$ from the observables $\mathbf{o}_n(t)$ and controllables $\mathbf{c}_n(t)$, allowing a decision-maker on node n to select the best control settings.

Elsewhere in this book, we rarely need this level of detailed notation. We therefore use the following shorthand: $m_k = f_k(\mathbf{o}, \mathbf{s})$, or simply $m = f(\mathbf{o}, \mathbf{s})$. Likewise, $\tilde{U}(\mathbf{s}_i)$ is shorthand for the utility of candidate strategy $\mathbf{s}_i$.

Unlike more common notations, where $y = f(x)$, we explicitly distinguish between observables and controllables to expose the system's ability to make decisions. This notation is similar to that of MDPs, where the reward is expressed as a function of state and action: $U = R(\mathbf{s}, \mathbf{a})$, in Section 6.1.3.

[†] Weights do not need to be scalar values; lambda functions may be appropriate for some metrics.